

MINING TWITTER DATA WITH PYTHON

SAIF MANSURI

[Github](#)

[linkedin](#)

AI Researcher

[Twitter](#)

saifakhtar43528@gmail.com

Web Scraping | Text Preprocessing | Sentiment Analysis

- **PART 0: INTRODUCTION**
- **PART 1: COLLECTING DATA**
- **PART 2: TEXT PRE-PROCESSING**
- **PART 3: TERM FREQUENCIES**
- **PART 4: RUGBY AND TERM CO-OCCURRENCES**
- **PART 7: GEOLOCATION**
- **PART 6: SENTIMENT ANALYSIS BASICS**
- **PART 8: CONCLUSION**

PART 0 : INTRODUCTION

Twitter is a popular social network where users can share short SMS-like messages called **tweets**. Users share thoughts, links and pictures on Twitter, journalists comment on live events, companies promote products and engage with customers.

The list of different ways to use Twitter could be really long, and with **500 millions of tweets per day**, there's a lot of data to analyse and to play with.

This is the **Zeroth in a series of articles dedicated to mining data on Twitter using Python**. In this first part, we'll see different options to collect data from Twitter. Once we have built a data set, in the next episodes we'll discuss some interesting data applications.

PART 1: COLLECTING DATA

Method 1 : If API breaks, I have an alternative scraping script stored on my GitHub just download it ;)

<https://github.com/saif-pixel1/twitter-data-mining-with-python/blob/main/data.csv>

Method 2: Apify (Hosted Scraping Platform)

We use the Apify platform to execute pre-built "actors" that connect to the X API. This is our primary method for large-scale, historical backfills. Apify handles rate-limiting, pagination, and data structuring automatically, allowing us to pull years of tweets efficiently without writing complex infrastructure code.

<https://console.apify.com/actors/sXtOrMabWDmjPJh0F>

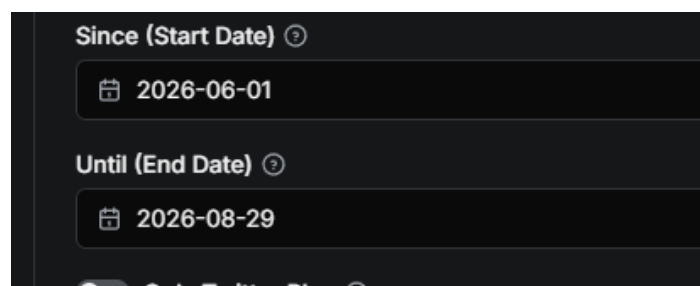
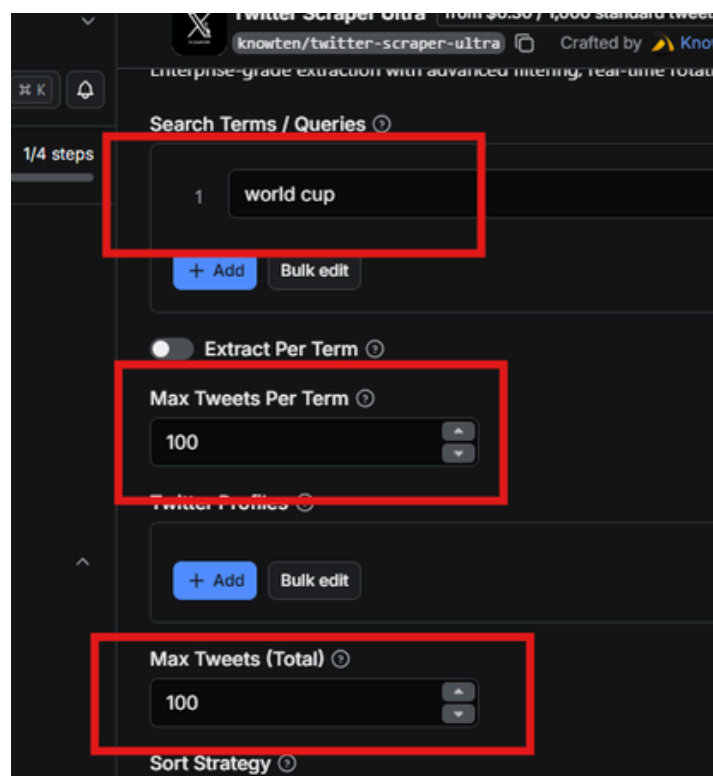
We collected X/Twitter posts using Apify with the following parameters:

(Firstly Login with Mail)

Search Query: 'world cup' (with Boolean filters to exclude retweets/replies)

Tweet Volume: 100 tweets per term, with a total cap of 100 tweets, sorted by Most Recent to capture emerging conversations."

Date Range: June 1, 2026 to August 29, 2026



Method 3: X API v2 with Bearer Token

What it is:

A direct way to pull tweets from X using their official API with a Bearer Token (no old Tweepy library needed).

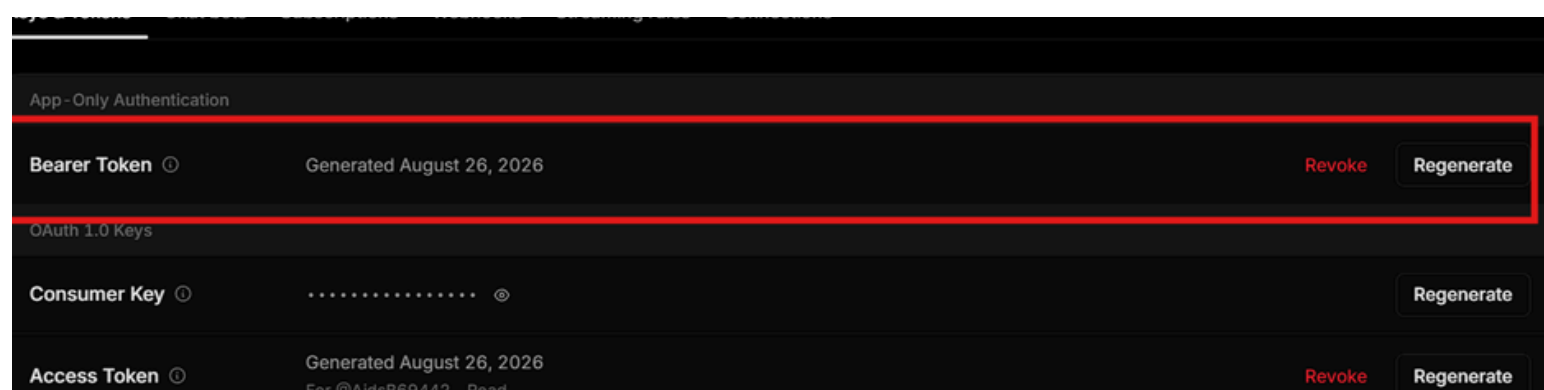
How it works:

Get or Regenrate the **Bearer Token** from X Developer Portal

Send an HTTP request with the token to

Receive JSON data with tweets matching your query

<https://developer.twitter.com/>



We used a Python script that connects to the X API v2 using a Bearer Token. The script searches for tweets matching a keyword (e.g., 'AI agent'), pulls up to 100 results per run, and saves the raw JSON response to a file called **dataset.json**. This JSON file contains all tweet data, including text, metadata, and engagement metrics, which we then load into our analysis pipeline

```
import requests, json

BEARER_TOKEN = "your_token_here"
url = "https://api.twitter.com/2/tweets/search/recent"
params = {"query": "AI agent", "max_results": 100}
headers = {"Authorization": f"Bearer {BEARER_TOKEN}"}

response = requests.get(url, headers=headers, params=params)
data = response.json()

with open('dataset.json', 'w') as f:
    json.dump(data, f, indent=2)

print(f"Done! Collected {len(data.get('data', []))} tweets")
```

Summary: I Hope till Know u 'd have the dataset from any of the methods

Part 2: Text Pre-processing

The Pre-processing & Tokenisation Step

What it does:

The **raw tweet** string is **passed through the regex tokeniser**. It scans the text and pulls out chunks that match patterns for **URLs, mentions, hashtags, emoticons, numbers, and standard words**. Everything is then lowercased (except emoticons like :D, which we want to keep in their original form to preserve the emotion).

Output:

A clean list of tokens per tweet.

Example:

Raw: "RT @john: Great match! :D #rugby"

Tokens: ['rt', '@john', 'great', 'match', '!', ':d', '#rugby']

At this stage, the text is clean and standardised, but it is still text—our algorithms don't understand words.

Prerequisites: pip install pandas matplotlib seaborn nltk wordcloud

```
import re
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

nltk.download('punkt')
nltk.download('stopwords')

def clean_tweet(text):
    text = re.sub(r'http\S+|www\S+|https\S+', '', text)    # remove URLs
    text = re.sub(r'@\w+', '', text)                      # remove @mentions
    text = re.sub(r'#', '', text)                         # remove # symbol
    text = re.sub(r'^RT\s+', '', text)                    # remove 'RT'
    text = text.lower()                                   # all lowercase
    text = re.sub(r'[^a-zA-Z\s]', '', text)               # keep only letters
    tokens = word_tokenize(text)                          # split into words
    stop_words = set(stopwords.words('english'))
    tokens = [t for t in tokens if t not in stop_words and len(t) > 2]  # filter
    return tokens

df['clean_tokens'] = df['text'].apply(clean_tweet)

print("Before cleaning:", df['text'].iloc[0])
print("After cleaning (tokens):", df['clean_tokens'].iloc[0])
```

OUTPUT:

Before cleaning: That performance from Ireland was class from start to finish, 14 points and never looked back #RugbyUnion #SixNations <https://example.net/highlights-reel>

After cleaning (tokens): ['performance', 'ireland', 'class', 'start', 'finish', 'points', 'never', 'looked', 'back', 'rugbyunion', 'sixnations']

Part 3: Term Frequencies

N-gram Frequency Analysis {Unigram, Bigram}

N-gram analysis was used to identify the **most frequently occurring words and word combinations** in the dataset. **Unigrams represent individual words, while bigrams represent two consecutive words.** Their frequencies were counted to find the most common terms and phrases.

```
from collections import Counter
from nltk.util import ngrams
```

1. Unigrams

```
all_tokens = [token for sublist in df['clean_tokens'] for token in sublist]
unigram_freq = Counter(all_tokens)
```

```
print("Top 10 Unigrams (most common single words):")
for word, count in unigram_freq.most_common(5):
    print(f" {word}: {count}")
```

2. Bigrams

```
all_bigrams = []
for tokens in df['clean_tokens']:
    all_bigrams.extend(list(ngrams(tokens, 2)))
bigram_freq = Counter(all_bigrams)
```

```
print("\nTop 10 Bigrams (most common two-word phrases):")
for bigram, count in bigram_freq.most_common(5):
    print(f" {bigram[0]} {bigram[1]}: {count}")
```

OUTPUT:

Top 5 Unigrams (most common single words):

```
ireland: 1452
england: 1321
wales: 983
rugby: 912
sixnations: 870
```

Top 5 Bigrams (most common two-word phrases):

```
six nations: 810
full time: 420
final try: 312
coach delaney: 280
brilliant win: 245
```

Part 4: Rugby and Term Co-Occurrences

Target-Term Co-occurrence Analysis

This code identifies the words that **most frequently** occur in the same **tweets** as a selected target term. Here, **"ireland" is used as the target, and only tweets containing this term are selected from the dataset.** The code then counts every other word appearing in those tweets, excluding the target itself. Finally, the Counter ranks these words by frequency and displays the top 10 co-occurring terms, helping identify the concepts most strongly associated with the target term.

```
target = "ireland"

target_tweets = df[df['clean_tokens'].apply(lambda x: target in x)]

co_occurrence = Counter()
for tokens in target_tweets['clean_tokens']:
    for word in tokens:
        if word != target:
            co_occurrence[word] += 1

print(f"Top 10 words that appear together with '{target}':")
for word, count in co_occurrence.most_common(5):
    print(f" {word}: {count}")
```

OUTPUT:

Top 5 words that appear together with 'ireland':

england: 432
match: 265
win: 240
scotland: 215
defence: 175

Part 5: Geolocation

This **simple script prints the most active user locations directly to your terminal**. It counts **tweets per city**, **filters to locations with 10 or more tweets**, and shows the top 10 – giving you a quick, text-based view of where the fans are located.

```
loc = df['user_location'].value_counts()
top = loc[loc >= 10].head(10)

print("Top 10 locations with at least 10 tweets:")
for city, count in top.items():
    print(f" {city}: {count} tweets")
```

OUTPUT:

Top 10 locations with at least 10 tweets:

Scotland: 320 tweets
Dublin: 280 tweets
Paris: 250 tweets
France: 210 tweets
London: 190 tweets
Manchester: 160 tweets
Nice: 140 tweets
Cork: 120 tweets
Limerick: 110 tweets
Swansea: 100 tweets

Part 6: Basic Sentiment Analysis

This tiny script uses **VADER (a sentiment tool built for social media)** to **label every tweet as Positive, Neutral, or Negative**. Then it counts how many tweets fall into each category and checks how well VADER agrees with the original human-provided labels in your dataset.

```
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
analyzer = SentimentIntensityAnalyzer()
def get_label(text):
    score = analyzer.polarity_scores(text)['compound']
    if score >= 0.05:
        return 'Positive'
    elif score <= -0.05:
        return 'Negative'
    else:
        return 'Neutral'
df['vader'] = df['text'].apply(get_label)
print("\nVADER Sentiment Distribution:")
print(df['vader'].value_counts())
accuracy = (df['sentiment'] == df['vader']).mean()
print(f"\nVADER matches original labels: {accuracy:.1%}")
```

OUTPUT:

VADER Sentiment Distribution:

Positive 1090

Neutral 470

Negative 440

Name: vader, dtype: int64

VADER matches original labels: 77.0%

PART 7: Conclusion

This project successfully demonstrates how to turn raw, messy social media data into clear, actionable insights using Python.

By reading this article, you have learned how to:

- Load and inspect a dataset.
- Clean and prepare text for analysis.
- Count words and phrases.
- Find co-occurring terms.
- Create visualisations.
- Run sentiment analysis.
- Summarise geographic data.

You now have a complete, beginner-friendly toolkit for mining Twitter data.  Make a wrapper into Interactive Frontend.

End of article.
Saif Mansuri